Practical no.7 : Regular Expressions and Multithreading

Part A — Regular Expressions

Q1. Write a Python program to check whether a given string contains a specific pattern using Regular Expressions.

Code

```
import re

text = input("Enter a String: ")
pattern = input("Enter Pattern to Search: ")

result = re.search(pattern, text)

if result:
    print("Pattern Found")
else:
    print("Pattern Not Found")
```

Sample Output

```
Enter a String: Python Programming
Enter Pattern to Search: Python
Pattern Found
```

Q2. Write a Python program to find all occurrences of a word in a given string using Regular Expressions.

Code

```
import re
```

```
text = "Python is easy. Python is powerful. Python is popular."

pattern = "Python"

matches = re.findall(pattern, text)

print("Occurrences:", matches)
print("Number of Occurrences:", len(matches))
```

Output

```
Occurrences: ['Python', 'Python', 'Python']
Number of Occurrences: 3
```

Q3. Write a Python program to validate a mobile number using Regular Expressions.

Code

```
import re

mobile = input("Enter Mobile Number: ")

pattern = r"^[6-9][0-9]{9}$"

if re.fullmatch(pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Output

```
Enter Mobile Number: 9876543210
Valid Mobile Number
```

Q4. Write a Python program to validate an email address using Regular Expressions.

Code

```
import re
```

```
email = input("Enter Email Address: ")

pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"

if re.fullmatch(pattern, email):
    print("Valid Email Address")
else:
    print("Invalid Email Address")
```

Q5. Write a Python program to extract all numbers from a given string using Regular Expressions.

Code

```
import re

text = "I have 10 apples, 25 oranges and 15 bananas."

numbers = re.findall(r"\d+", text)

print("Numbers Found:", numbers)
```

Output

```
Numbers Found: ['10', '25', '15']
```

Q6. Write a Python program to replace multiple spaces in a string with a single space using Regular Expressions.

Code

```
import re

text = "Python    is    a    programming    language."

result = re.sub(r"\s+", " ", text)

print(result)
```

Output

Python is a programming language.

Part B — Multithreading

Q7. Write a Python program to create and execute a thread.

Code

```
import threading

def display():
    for i in range(1, 6):
        print("Thread:", i)

t = threading.Thread(target=display)

t.start()

t.join()

print("Main Program Completed")
```

Q8. Write a Python program to create multiple threads and demonstrate their execution.

Code

```
import threading

def task(name):
    for i in range(1, 4):
        print(name, ":", i)

t1 = threading.Thread(target=task, args=("Thread 1",))
t2 = threading.Thread(target=task, args=("Thread 2",))

t1.start()
t2.start()
```

```
t1.join()
t2.join()

print("All Threads Completed")
```

Note: The order of output from the two threads may vary because they execute concurrently.

Part C — Thread Synchronization

Q9. Write a Python program to demonstrate a race condition when multiple threads access a shared variable.

Code

```
import threading

counter = 0

def increment():
    global counter

    for i in range(100000):
        counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Counter:", counter)
```

Purpose: This program demonstrates multiple threads working on the same shared resource.

Q10. Write a Python program to synchronize access to a shared resource using Lock.

Code

```
import threading

counter = 0

lock = threading.Lock()

def increment():
    global counter

    for i in range(100000):
        with lock:
            counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Counter:", counter)
```

Expected Output

Final Counter: 200000

Here, Lock ensures that only one thread modifies the shared variable at a time.

Q11. Write a Python program to demonstrate thread synchronization using Lock while accessing a shared bank account.

Code

```
import threading

balance = 1000

lock = threading.Lock()

def withdraw(amount, name):
    global balance

    with lock:
        if balance >= amount:
            print(name, "is withdrawing", amount)

            balance -= amount

            print("Remaining Balance:", balance)
        else:
            print(name, "cannot withdraw", amount)
            print("Insufficient Balance")

t1 = threading.Thread(
    target=withdraw,
    args=(700, "Thread 1")
)

t2 = threading.Thread(
    target=withdraw,
    args=(500, "Thread 2")
)

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Balance:", balance)
```

This is a good **real-world example of synchronization**.

Part D — Priority Queue with Multithreading

Q12. Write a Python program to create a Priority Queue and insert elements with different priorities.

Code

```
from queue import PriorityQueue

pq = PriorityQueue()

pq.put((3, "Low Priority"))
pq.put((1, "High Priority"))
pq.put((2, "Medium Priority"))

while not pq.empty():
    priority, task = pq.get()

    print("Priority:", priority, "Task:", task)
```

Output

```
Priority: 1 Task: High Priority
Priority: 2 Task: Medium Priority
Priority: 3 Task: Low Priority
```

Important: In Python's PriorityQueue, a **smaller priority number is processed first**.

Q13. Write a Python program to demonstrate Priority Queue behavior using multiple threads.

Code

```
import threading
from queue import PriorityQueue

pq = PriorityQueue()
```

```
def add_tasks():
    pq.put((3, "Task C"))
    pq.put((1, "Task A"))
    pq.put((2, "Task B"))

def process_tasks():
    while not pq.empty():
        priority, task = pq.get()

        print(
            "Processing:",
            task,
            "Priority:",
            priority
        )

t1 = threading.Thread(target=add_tasks)
t2 = threading.Thread(target=process_tasks)

t1.start()
t1.join()

t2.start()
t2.join()

print("All Tasks Completed")
```

Q14. Write a Python program to implement a thread-safe producer-consumer system using Queue.

Code

```
import threading
import queue
import time

q = queue.Queue()

def producer():
    for i in range(1, 6):
        print("Producing:", i)
```

```
        q.put(i)

        time.sleep(0.5)

def consumer():
    for i in range(1, 6):
        item = q.get()

        print("Consuming:", item)

        q.task_done()

producer_thread = threading.Thread(target=producer)
consumer_thread = threading.Thread(target=consumer)

producer_thread.start()
consumer_thread.start()

producer_thread.join()
consumer_thread.join()

print("Processing Completed")
```

This is useful for explaining how a queue can safely coordinate data between threads.

Additional Practice

Q15. Write a Python program to validate a password using Regular Expressions.

Conditions:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit

Code

```
import re
```

```
password = input("Enter Password: ")

pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$"

if re.fullmatch(pattern, password):
    print("Valid Password")
else:
    print("Invalid Password")
```

Q16. Write a Python program to extract dates from a given text using Regular Expressions.

Code

```
import re

text = """
Exam dates are 10-09-2026 and 25-09-2026.
The practical exam is on 05-10-2026.
"""

dates = re.findall(r"\d{2}-\d{2}-\d{4}", text)

print("Dates Found:")

for date in dates:
    print(date)
```